

Assignment IV

DATA471

Name: Hamid Mehmood Student ID: 300693052

2026-09-20

Contents

Exercise 1	1
Part 1: Which sample quantiles should we use? (10%)	1
Part 2: A bootstrap bias-corrected IQR estimator (20%)	2
Part 3: Monte Carlo comparison (20%)	3
Exercise 2	7
The dataset	7
Q1: What is the frequency of the series?	7
Q2: Overall behaviour and long-term trend	8
Q3: Is there seasonality?	9
Q4: Is the variability constant? Would a transformation help?	10
Q5: Any unusual observations?	11
Q6: Seasonal plot	12
Q7: Seasonal subseries plot	13
Q8: Would you detrend or transform before modelling?	13

Exercise 1

Part 1: Which sample quantiles should we use? (10%)

To compute the IQR estimator, $\hat{\lambda}_{\text{IQR}} = \ln 3 / (q_{3/4} - q_{1/4})$, I need the first and third quartiles of my sample. That sounds like a non-issue, but it turns out there is no single “correct” way to pull a quartile out of a finite sample. Wicklin (2017) goes through nine different definitions that all show up in real software, and they can genuinely disagree on the same data. So before I trust my quartiles, I should know which definition I am actually using.

Summary of Wicklin (2017)

Wicklin’s post is basically a friendly walk through the nine quantile definitions that Hyndman and Fan (1996) catalogued (I will call them H and F). Here is what I took from it.

The basic idea. Sort the sample $x_{(1)} \leq \dots \leq x_{(N)}$. To estimate the p -th quantile you find the two neighbouring order statistics that sit either side of it, and then decide what to do in the gap between them. Every one of the nine methods can be written in the same general form,

$$q = (1 - \lambda)x_{(j)} + \lambda x_{(j+1)}, \quad j = \lfloor Np + m \rfloor,$$

where $\lambda \in [0, 1]$ is an interpolation weight and m is a small constant that changes with the method. So the nine “definitions” are not nine unrelated ideas; they are nine choices of m and of how you set λ .

Two families. Wicklin splits them into two groups. Types 1 to 3 are *discrete* (rounding) methods: the answer is always an actual data value (or the average of two), so the quantile function looks like a staircase. Types 4 to 9 are *continuous* (interpolation) methods: they join the points up with straight lines, so the answer can be anything between two observations.

Where they disagree. The methods differ most for small samples and out in the tails (or wherever there is a big gap between neighbouring values). Wicklin shows that for a sample of 100 the nine methods basically land on top of each other. His honest summary is “yes and no”: the choice matters in theory and for tiny samples, but hardly at all for large ones. Either way, his advice is to always say which method you used.

Software defaults. This is the part that catches people out. SAS supports five of the nine (the QNTLDEF= option) and defaults to type 2, a discrete method, while R defaults to type 7, a continuous one. Same data, different quartiles.

Which definition would I use, and why?

I will use a continuous method throughout, specifically **type 8**. My reasons, in plain terms:

1. My estimator depends on a *spread*, $q_{3/4} - q_{1/4}$, not on a single order statistic. Interpolation uses both of the bracketing points, so it gives a smoother spread than the discrete methods, which can snap both quartiles onto the same repeated value.
2. Among the continuous methods, type 8 is the one Hyndman and Fan actually recommend, because it is approximately median-unbiased whatever the underlying distribution is. Since the very next part of this exercise is about bias, it feels right to start from the quartile rule with the best bias behaviour.
3. Being honest, for $n = 100$ the choice barely matters, but for the $n = 5$ and $n = 30$ cases coming up it really does, so it is worth picking on purpose rather than by accident.

In R this is just `quantile(x, type = 8)`. Type 7 (R’s default) would be fine too and would not change much for larger n .

What is the default in R and Python?

- **R:** `quantile()` defaults to **type 7** (continuous, linear interpolation). You change it with the `type` argument.
- **Python:** `numpy` and `pandas` also default to linear interpolation, which is **the same as R’s type 7** (`method = "linear"` in current NumPy).
- **SAS (for contrast):** defaults to **type 2**, a discrete method, which is exactly why SAS and R/Python can give different quartiles on a small sample.

Part 2: A bootstrap bias-corrected IQR estimator (20%)

Deriving $\tilde{\lambda}$

The bias of the IQR estimator is

$$\text{Bias}(\hat{\lambda}_{\text{IQR}}) = \mathbb{E}_{\lambda}[\hat{\lambda}_{\text{IQR}}(\mathbf{X})] - \lambda.$$

I cannot just compute this, for two reasons: it depends on the true λ (which is the thing I am trying to estimate), and the sampling distribution of a ratio of quartiles has no nice closed form. This is exactly the situation the bootstrap is built for.

The bootstrap idea is to treat my observed sample as if it were the whole population, and to draw new samples from it by resampling **with replacement**. Inside this “bootstrap world” I know everything, so I can measure the bias directly. The value I actually computed, $\hat{\lambda}_{\text{IQR}}(\mathbf{x})$, plays the role of the truth, and I estimate the bias by Monte Carlo:

$$\widehat{\text{Bias}} = \underbrace{\frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{\text{IQR}}(\mathbf{X}_b^*)}_{\text{mean over } B \text{ resamples}} - \hat{\lambda}_{\text{IQR}}(\mathbf{x}), \quad \mathbf{X}_b^* \text{ drawn with replacement from } \mathbf{x}.$$

The first-order bias-corrected estimator just subtracts this estimated bias:

$$\tilde{\lambda} = \hat{\lambda}_{\text{IQR}}(\mathbf{x}) - \widehat{\text{Bias}} = 2 \hat{\lambda}_{\text{IQR}}(\mathbf{x}) - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{\text{IQR}}(\mathbf{X}_b^*)$$

As instructed, I fix $B = 200$ resamples per input sample.

Implementation

Here are the three estimators. I use R's built-in `quantile(type = 8)` throughout, so the code stays as close to the formulas as possible.

```
lambda_ml <- function(x) 1 / mean(x)

lambda_iqr <- function(x) {
  qs <- quantile(x, probs = c(0.25, 0.75), type = 8, names = FALSE)
  log(3) / (qs[2] - qs[1])
}

lambda_tilde <- function(x, B = 200) {
  n <- length(x)
  lhat <- lambda_iqr(x)
  boot <- replicate(B, lambda_iqr(sample(x, n, replace = TRUE)))
  fail <- sum(!is.finite(boot))           # resamples whose IQR came out zero
  boot <- boot[is.finite(boot)]
  list(est = 2 * lhat - mean(boot),      # = lhat - estimated bias
       fail = fail)
}
```

The `fail` value in that return is there for a concrete reason. When n is very small, a bootstrap resample can repeat values so that its first and third quartiles come out equal, which gives a zero IQR and an infinite estimate. The `is.finite` line drops those degenerate resamples before averaging so that $\tilde{\lambda}$ stays finite, and `fail` records how many had to be dropped. That count is essentially zero for moderate samples but noticeable at $n = 5$, so keeping it gives me a real handle on why the estimator becomes unreliable for tiny samples, which I return to in Part 3.

As a quick check that the correction pushes in the right direction: the ML estimator of an exponential rate has a known bias, $\mathbb{E}[\hat{\lambda}_{\text{ML}}] = \frac{n}{n-1}\lambda$, so it overestimates for small n , and the IQR estimator overestimates too. A good bias correction should pull $\tilde{\lambda}$ back down towards λ on average, which is what happens in Part 3.

Part 3: Monte Carlo comparison (20%)

Design

I compare the three estimators on the grid $\Lambda \times N$ with $\lambda \in \{0.1, 1, 10\}$ and $n \in \{5, 30, 100\}$, using **mean squared error** as the only metric, $\text{MSE}(\hat{\lambda}) = \mathbb{E}[(\hat{\lambda} - \lambda)^2]$, estimated by averaging $(\hat{\lambda} - \lambda)^2$ over R independent replications.

How many replications, and why λ barely matters

There is a nice structural fact that basically decides this for me. All three estimators are **scale-equivariant**: if I rescale the data $x \mapsto cx$, the estimate becomes $\hat{\lambda}/c$ (true for the mean, for the quartile gap, and so for $\tilde{\lambda}$ too). Since $X \sim \text{Exp}(\lambda)$ means $\lambda X \sim \text{Exp}(1)$, the distribution of the relative error $\hat{\lambda}/\lambda$ does not depend on λ at all. So

$$\text{MSE}(\hat{\lambda}; \lambda) = \lambda^2 \text{MSE}(\hat{\lambda}; 1),$$

the MSE is just λ^2 times a number that depends only on n and the estimator.

Two things follow. First, I do not need more replications for different λ ; the same R gives the same relative precision everywhere. Second, I can use this as a free check: the columns of MSE/λ^2 should match across λ .

For the size of R , I follow Robert and Casella (2009) and treat the estimated MSE as a sample mean, watching its Monte Carlo standard error, $\text{SE} = \text{sd}((\hat{\lambda} - \lambda)^2)/\sqrt{R}$. I use $R = 3000$ for every cell. That gives standard errors of only a couple of percent of the MSE for $n = 30$ and $n = 100$, which is plenty. The one exception is $n = 5$ for the IQR-based estimators, whose squared error is very heavy-tailed. There, no realistic R settles the estimate down, and the honest takeaway is not “run more replications” but “do not use these estimators at $n = 5$ ” (more on that below).

Running the experiment

The loop below calls the three estimators directly. Each $\tilde{\lambda}$ does 200 bootstrap resamples, so the whole run takes a few minutes; the chunk is cached so re-knitting is instant.

```
Lambda <- c(0.1, 1, 10); Ns <- c(5, 30, 100); R <- 3000
set.seed(301471)

rows <- list(); k <- 0
for (lam in Lambda) for (n in Ns) {
  ml <- iqr <- til <- fails <- numeric(R)
  for (r in 1:R) {
    x <- rexp(n, rate = lam)
    ml[r] <- lambda_ml(x)
    iqr[r] <- lambda_iqr(x)
    tt <- lambda_tilde(x); til[r] <- tt$est; fails[r] <- tt$fail
  }
  mse <- function(e) mean((e - lam)^2)
  se <- function(e) sd((e - lam)^2) / sqrt(R)
  k <- k + 1
  rows[[k]] <- data.frame(
    lambda = lam, n = n,
    MSE_ML = mse(ml), MSE_IQR = mse(iqr), MSE_TIL = mse(til),
    seML = se(ml), seIQR = se(iqr), seTIL = se(til),
    biasML = mean(ml) - lam, biasIQR = mean(iqr) - lam, biasTIL = mean(til) - lam,
    varIQR = var(iqr), varTIL = var(til), meanfail = mean(fails))
}
res <- do.call(rbind, rows)
```

Results

```
tab <- data.frame(
  lambda = res$lambda, n = res$n,
  MSE_ML = signif(res$MSE_ML, 3),
  MSE_IQR = signif(res$MSE_IQR, 3),
  MSE_tilde = signif(res$MSE_TIL, 3))
knitr::kable(tab, caption = "MSE of the three estimators over the grid (R = 3000).")
```

Table 1: MSE of the three estimators over the grid ($R = 3000$).

lambda	n	MSE_ML	MSE_IQR	MSE_tilde
0.1	5	5.66e-03	1.06e-02	1.17e+00
0.1	30	3.79e-04	9.82e-04	1.19e-03

lambda	n	MSE_ML	MSE_IQR	MSE_tilde
0.1	100	1.07e-04	2.43e-04	2.87e-04
1.0	5	7.05e-01	1.56e+00	3.74e+03
1.0	30	4.03e-02	9.67e-02	1.19e-01
1.0	100	1.09e-02	2.41e-02	2.81e-02
10.0	5	6.06e+01	1.21e+02	8.11e+04
10.0	30	4.07e+00	1.02e+01	1.26e+01
10.0	100	1.09e+00	2.33e+00	2.68e+00

```
rel <- data.frame(
  lambda = res$lambda, n = res$n,
  rel_ML = signif(res$MSE_ML / res$lambda^2, 3),
  rel_IQR = signif(res$MSE_IQR / res$lambda^2, 3),
  rel_tilde = signif(res$MSE_TIL / res$lambda^2, 3))
knitr::kable(rel, caption = "Relative MSE = MSE /  $\lambda^2$ . For each n these should agree across lambda")
```

Table 2: Relative MSE = MSE / λ^2 . For each n these should agree across lambda (scale-equivariance), and they do for n = 30 and 100. The n = 5 tilde column does not, which reveals its instability.

lambda	n	rel_ML	rel_IQR	rel_tilde
0.1	5	0.5660	1.0600	1.17e+02
0.1	30	0.0379	0.0982	1.19e-01
0.1	100	0.0107	0.0243	2.87e-02
1.0	5	0.7050	1.5600	3.74e+03
1.0	30	0.0403	0.0967	1.19e-01
1.0	100	0.0109	0.0241	2.81e-02
10.0	5	0.6060	1.2100	8.11e+02
10.0	30	0.0407	0.1020	1.26e-01
10.0	100	0.0109	0.0233	2.68e-02

```
agg <- aggregate(cbind(MSE_ML/lambda^2, MSE_IQR/lambda^2, MSE_TIL/lambda^2) ~ n,
  data = res, FUN = mean)
names(agg) <- c("n", "ML", "IQR", "tilde")
matplot(agg$n, agg[, -1], type = "b", log = "y", pch = 19, lty = 1,
  col = c("#1b7837", "#2166ac", "#b2182b"),
  xlab = "sample size n", ylab = "relative MSE (log scale)",
  main = "Relative MSE vs n")
legend("topright", legend = c("ML", "IQR", "tilde (bias-corrected)"),
  col = c("#1b7837", "#2166ac", "#b2182b"), pch = 19, lty = 1, bty = "n")
```

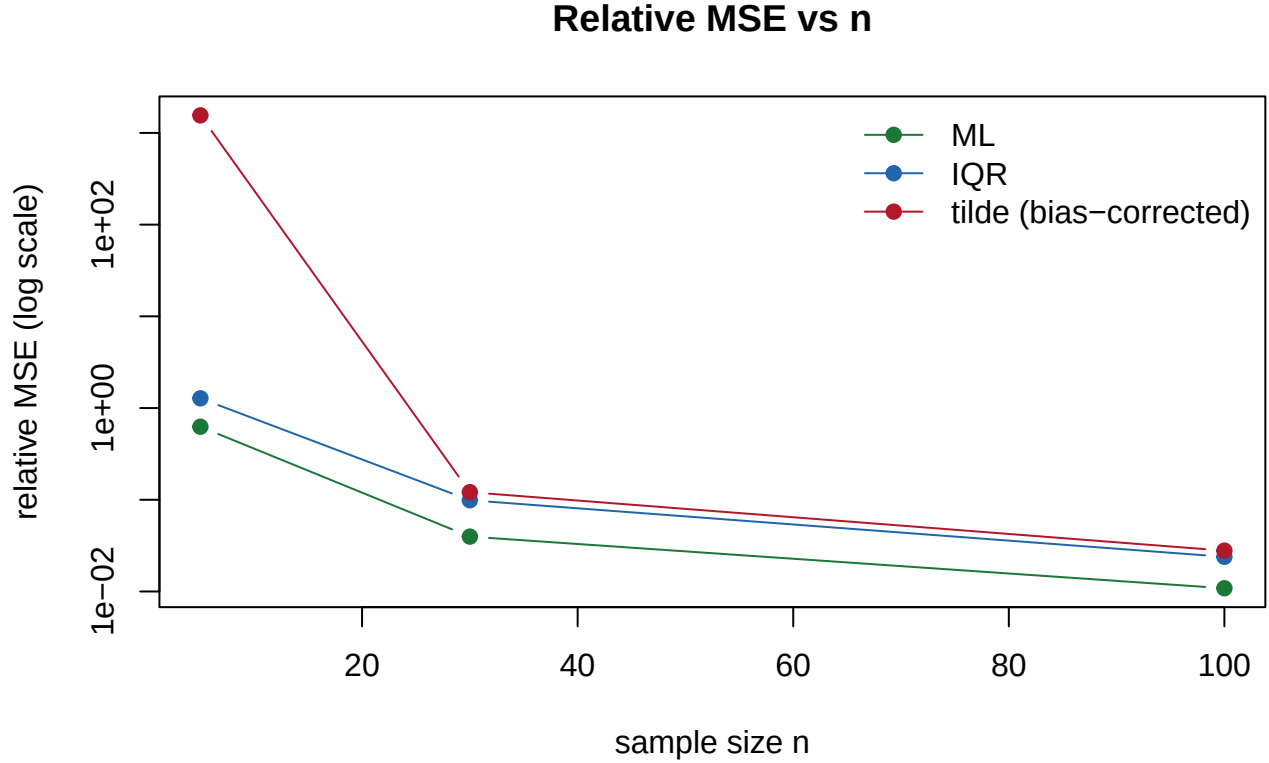


Figure 1: Relative MSE (MSE / λ^2 , averaged over the three lambda) versus n on a log scale. Lower is better. ML is best, the bootstrap correction sits just above plain IQR, and the tilde estimator at $n = 5$ is off the charts.

```
bt <- data.frame(
  lambda = res$lambda, n = res$n,
  bias_IQR = signif(res$biasIQR, 3),
  bias_tilde = signif(res$biasTIL, 3),
  var_IQR = signif(res$varIQR, 3),
  var_tilde = signif(res$varTIL, 3),
  avg_fails = signif(res$meanfail, 3))
knitr::kable(bt, caption = "Bias and variance of the IQR estimator vs its bias-corrected version, plus "
```

Table 3: Bias and variance of the IQR estimator vs its bias-corrected version, plus the average number of degenerate (zero-IQR) bootstrap resamples per sample (out of $B = 200$).

lambda	n	bias_IQR	bias_tilde	var_IQR	var_tilde	avg_fails
0.1	5	0.02200	-2.24e-01	1.02e-02	1.12e+00	0.328
0.1	30	0.00656	-2.36e-03	9.39e-04	1.18e-03	0.000
0.1	100	0.00174	-5.75e-04	2.40e-04	2.87e-04	0.000
1.0	5	0.25700	-3.69e+00	1.49e+00	3.72e+03	0.320
1.0	30	0.05830	-3.22e-02	9.33e-02	1.18e-01	0.000
1.0	100	0.01630	-7.93e-03	2.38e-02	2.80e-02	0.000
10.0	5	2.84000	-2.60e+01	1.13e+02	8.04e+04	0.314

lambda	n	bias_IQR	bias_tilde	var_IQR	var_tilde	avg_fails
10.0	30	0.62400	-2.82e-01	9.77e+00	1.25e+01	0.000
10.0	100	0.18000	-5.43e-02	2.30e+00	2.68e+00	0.000

What the experiment tells us

1. ML wins on MSE everywhere. Since the exponential model is correct for the simulated data, this is exactly what I would expect: ML uses the whole sample and is (asymptotically) efficient, while the IQR estimator throws away everything except two quartiles. Roughly, the IQR estimator carries about 2 to 2.5 times the MSE of ML for $n = 30$ and $n = 100$.

2. Scale-equivariance holds, a nice built-in check. For $n = 30$ and $n = 100$ the relative-MSE table is basically constant across λ (for example ML is about 0.040 at $n = 30$ and about 0.011 at $n = 100$ for every λ). That both reassures me the code is right and backs up using the same R for all λ .

3. The bootstrap correction reduces bias but not MSE. This is the most interesting bit. Comparing the plain IQR estimator with $\tilde{\lambda}$ in the bias table, the correction does shrink the bias: for $n = 100$, $\lambda = 1$ it moves from about +0.016 to about -0.008, even flipping sign, which is the slight overshoot that first-order corrections are known for. But the variance goes up (each bootstrap average adds its own noise), and since $\text{MSE} = \text{bias}^2 + \text{variance}$ with variance dominating here, $\tilde{\lambda}$ actually ends up with a slightly larger MSE than the plain IQR estimator. It is a clean example of the bias-variance trade-off: fixing the bias is not free.

4. $n = 5$ is a cautionary tale. At the smallest sample size everything is poor, but the IQR-based estimators are genuinely unreliable. Because a resample is drawn from only five points, the bootstrap frequently hits ties or near-ties, which throw up huge estimates and give $\tilde{\lambda}$ an essentially heavy-tailed, near-infinite-variance distribution. The failure column in the table above makes this concrete: at $n = 5$ a nonzero number of resamples per sample collapse to a zero IQR, while at $n = 30$ and $n = 100$ essentially none do. Its estimated MSE at $n = 5$ jumps around a lot and its Monte Carlo standard error is about the same size as the estimate itself, so those particular numbers should be read as “very large and unstable” rather than taken literally. The practical message: the IQR estimator, and especially its bootstrap correction, needs a decent sample size (around $n = 30$ or more) to behave.

Bottom line. If I trust the exponential model, I would use the ML estimator. The IQR estimator is a reasonable, simple backup for moderate n , and it would really earn its keep if I were worried about outliers or contamination (which is the whole point of a robust estimator, though I have not tested that here). The bootstrap bias correction does what it promises for bias, but it is not a free win for MSE, and it should not be used at very small n .

Exercise 2

```
library(fpp2) # the ausbeer dataset comes from this package
```

The dataset

The **ausbeer** series is the quarterly beer production of Australia, measured in megalitres. It is one of the example series Hyndman and Athanasopoulos (2018) use in their textbook, so it is a good series to practise looking at before doing any modelling. Everything below is plain exploratory data analysis: I look at the series, describe what I see, and back each claim up with a plot.

Q1: What is the frequency of the series?

The frequency is **4**: there are four observations per year, one per quarter. There are two easy ways to see this.

```
frequency(ausbeer)
```

```
## [1] 4
```

```
c(start = paste(start(ausbeer), collapse = " Q"),  
  end   = paste(end(ausbeer),   collapse = " Q"))
```

```
##      start      end  
## "1956 Q1" "2010 Q2"
```

First, the series is stored as a quarterly time series that runs from 1956 Q1 to 2010 Q2, so its `frequency` attribute is 4. Second, and more importantly, you can read it straight off a plot. If I zoom in on the first few years, the pattern repeats itself every four observations, with a clear tall peak once a year and a dip in between.

```
plot(window(ausbeer, end = c(1962, 4)), ylab = "megalitres", xlab = "year",  
      main = "First few years of ausbeer", col = "#2166ac")  
points(window(ausbeer, end = c(1962, 4)), pch = 19, cex = 0.6, col = "#2166ac")
```

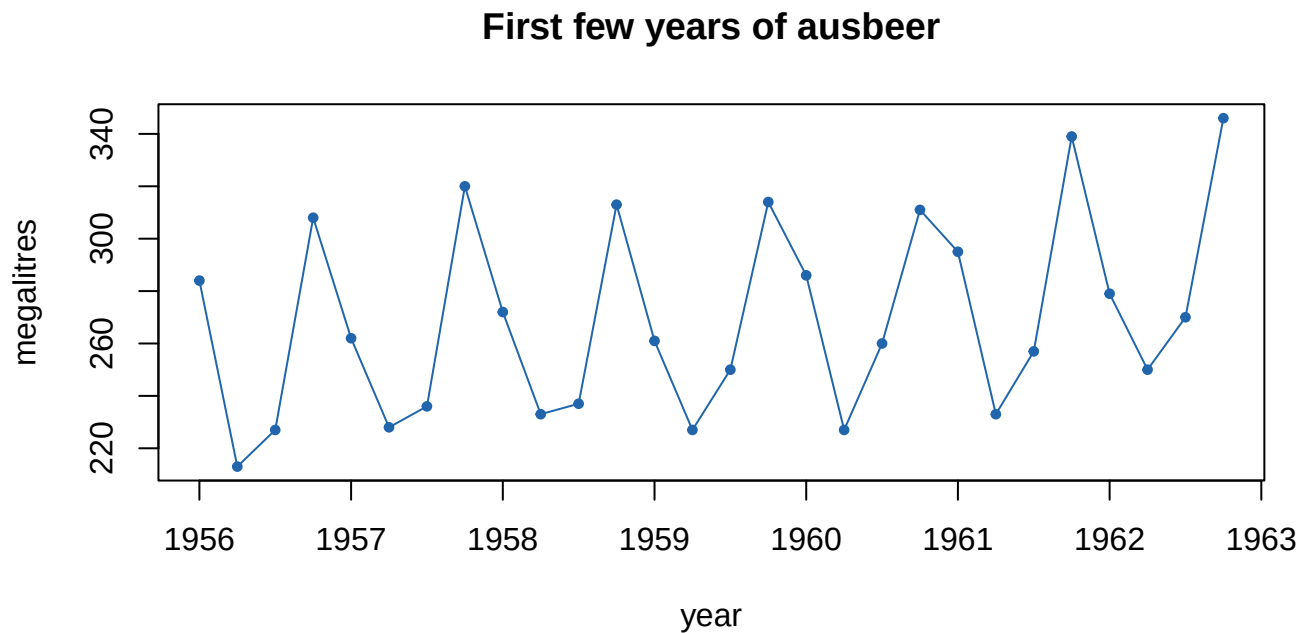


Figure 2: Zooming in on 1956 to 1962: the pattern repeats every four quarters, so the frequency is 4.

Counting the points between one peak and the next gives four, which is the frequency.

Q2: Overall behaviour and long-term trend

```
plot(ausbeer, ylab = "megalitres", xlab = "year",  
      main = "Quarterly Australian beer production, 1956 to 2010", col = "#2166ac")
```


Quarterly Australian beer production, 1956 to 2010

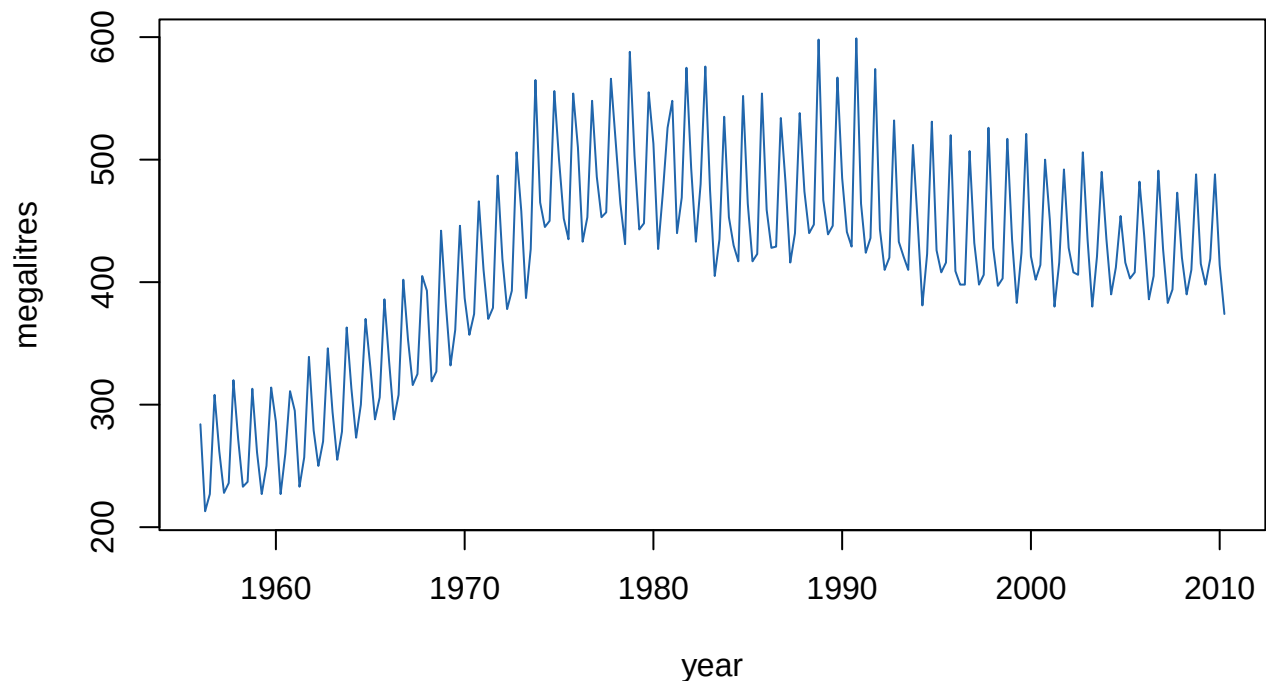


Figure 3: The full ausbeer series. A strong rise into the mid-1970s, a long plateau, then a gradual decline.

Yes, there is a clear long-term trend, but it is not a single straight line. Production rises steeply from about 1956 to the mid-1970s, roughly doubling from around 250 to over 500 megalitres. It then flattens into a long plateau through the late 1970s, 1980s and early 1990s, with the highest peaks of the whole series in this stretch (up to 599). After the mid-1990s it drifts gently downwards to the end of the series. So the trend goes up, levels off, and then comes down again. On top of this slow movement there is a very regular saw-tooth wobble, which is the seasonal pattern I look at next.

Q3: Is there seasonality?

Yes, and it is strong and very regular. The quickest way to summarise it is the average production in each quarter across all years:

```
round(tapply(ausbeer, cycle(ausbeer), mean), 1) # 1 = Q1, 2 = Q2, 3 = Q3, 4 = Q4
```

```
##      1      2      3      4
## 416.8 372.6 387.4 485.4
```

The pattern within a year is: production is high in Q1, drops to its lowest in Q2, recovers a little in Q3, and then jumps to its highest in Q4. In other words Q4 is the big quarter and Q2 is the quiet one. This makes sense for Australia, where Q4 covers the run-up to the warm Christmas and summer season in the Southern hemisphere. The seasonal swing is large, roughly 100 megalitres between the low and high quarters, which is not small next to a level of around 400 to 500. I look at this seasonal shape in detail in Q6 and Q7.

Q4: Is the variability constant? Would a transformation help?

To judge the variability I plotted the size of the seasonal swing (the largest minus the smallest value within each year) over time.

```
yrs <- unique(floor(time(ausbeer)))
m <- matrix(NA, length(yrs), 4, dimnames = list(yrs, c("Q1","Q2","Q3","Q4")))
for (i in seq_along(ausbeer))
  m[as.character(floor(time(ausbeer)[i])), cycle(ausbeer)[i]] <- ausbeer[i]
full <- m[rowSums(is.na(m)) == 0, ]
amp <- apply(full, 1, max) - apply(full, 1, min)
plot(as.numeric(rownames(full)), amp, type = "b", pch = 19, col = "#b2182b",
      xlab = "year", ylab = "within-year range", main = "Seasonal amplitude over time")
```

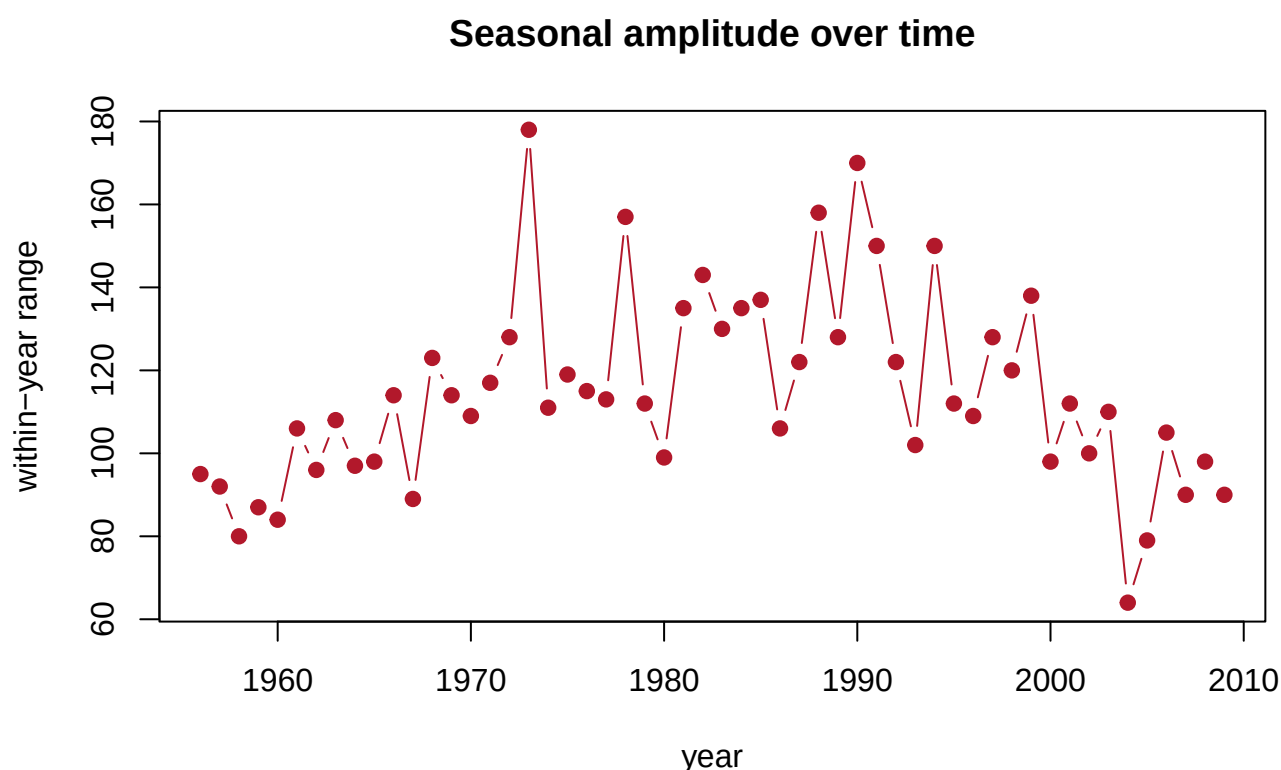


Figure 4: Within-year range (max minus min) over time. Larger in the high-production decades, smaller when the level is low.

The variability is not perfectly constant, but the story is more subtle than “it grows with the level”. The swing is around 90 in the low-production 1950s, grows to about 150 to 175 in the high-production 1970s to early 1990s, and then shrinks back to roughly 65 to 105 in the 2000s. The interesting part is the end: production in the 2000s is still fairly high (around 400 to 490), yet the swings are as small as they were at the very start. So the amplitude tracks the era more than it tracks the level. A variance-stabilising transformation such as a log or a Box-Cox assumes the swing grows in proportion to the level, and that assumption does not really hold here (the recent years break it). So I would say a transformation is optional at best, and I would not reach for a strong one. The variability is close enough to constant that the trend and seasonality are the features worth worrying about.

Q5: Any unusual observations?

Most of the series is extremely regular, so “unusual” here means points that sit clearly off the smooth trend and their usual seasonal level. To find them I removed a centred moving average (a simple descriptive smoother, not a fitted model) and the quarterly averages, and looked at what was left.

```
ma <- stats::filter(ausbeer, c(1/8, rep(1/4, 3), 1/8), sides = 2) # trend-cycle
irr <- (ausbeer - ma) - tapply(ausbeer - ma, cycle(ausbeer), mean, na.rm = TRUE)[cycle(ausbeer)]
lab <- paste0(floor(time(ausbeer)), " Q", cycle(ausbeer))
top <- order(abs(irr), decreasing = TRUE)[1:4]
data.frame(quarter = lab[top], value = as.numeric(ausbeer)[top],
           deviation = round(irr[top], 1))
```

quarter	value	deviation
1981 Q1	548	49.6
1990 Q4	599	49.0
1988 Q4	598	41.2
1978 Q3	431	-38.5

```
plot(ausbeer, ylab = "megalitres", xlab = "year", col = "grey55",
     main = "Unusual observations")
points(time(ausbeer)[top], ausbeer[top], pch = 19, col = "#b2182b")
text(time(ausbeer)[top], ausbeer[top], lab[top], pos = 3, cex = 0.7, col = "#b2182b")
```

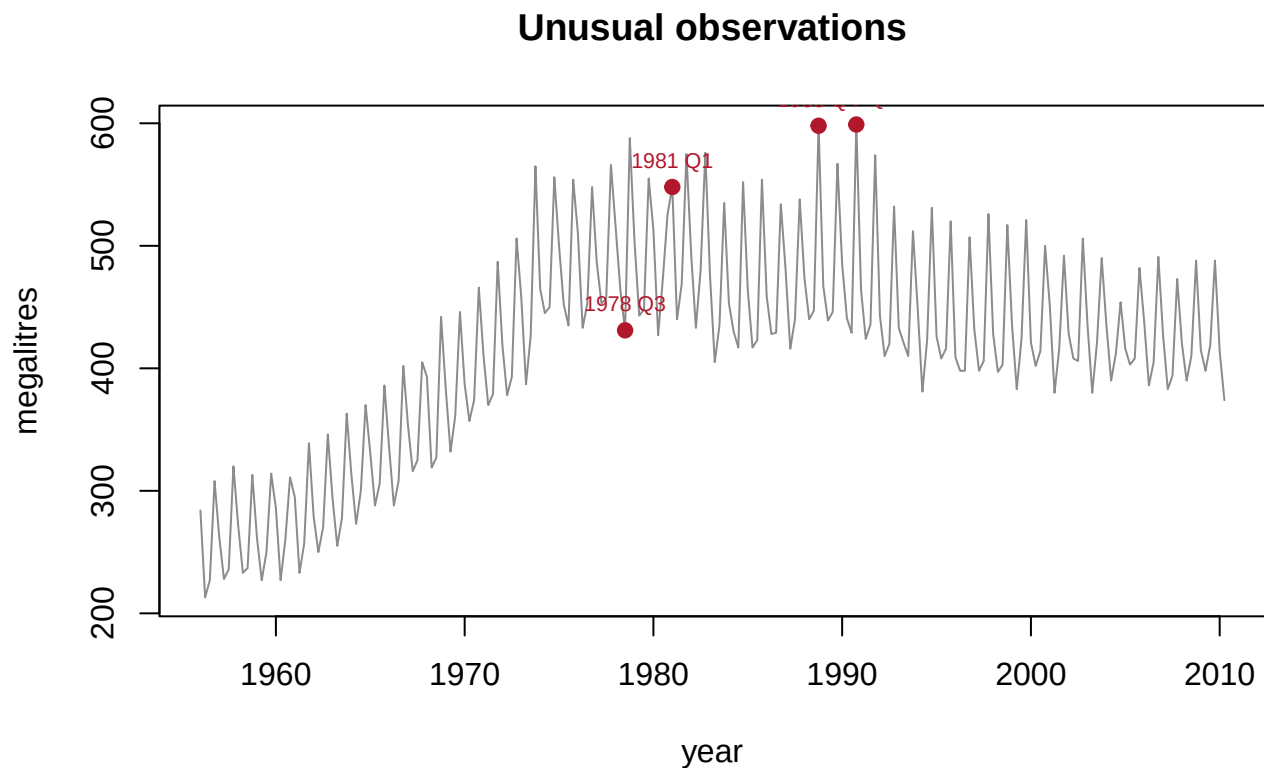


Figure 5: The series with the most unusual points marked in red.

A few points stand out. The two highest observations of the entire series, 1988 Q4 (598) and 1990 Q4 (599), are spikes that sit noticeably above even the other strong Q4 values around them. In the other direction, 1981 Q1 (548) is a strikingly strong first quarter, almost at the level of a typical Q4, and 1980 Q4 (526) is an unusually weak fourth quarter, well below the Q4 peaks on either side. They attract attention because each one departs from the very steady trend-plus-season pattern that the rest of the series follows so closely. None of them looks like a data error, they are just the years that wobbled a bit more than usual.

Q6: Seasonal plot

```
matplot(1:4, t(m), type = "l", lty = 1,  
        col = grey(seq(0.15, 0.85, length = nrow(m))),  
        xaxt = "n", xlab = "quarter", ylab = "megalitres",  
        main = "Seasonal plot (one line per year)")  
axis(1, at = 1:4, labels = c("Q1", "Q2", "Q3", "Q4"))
```

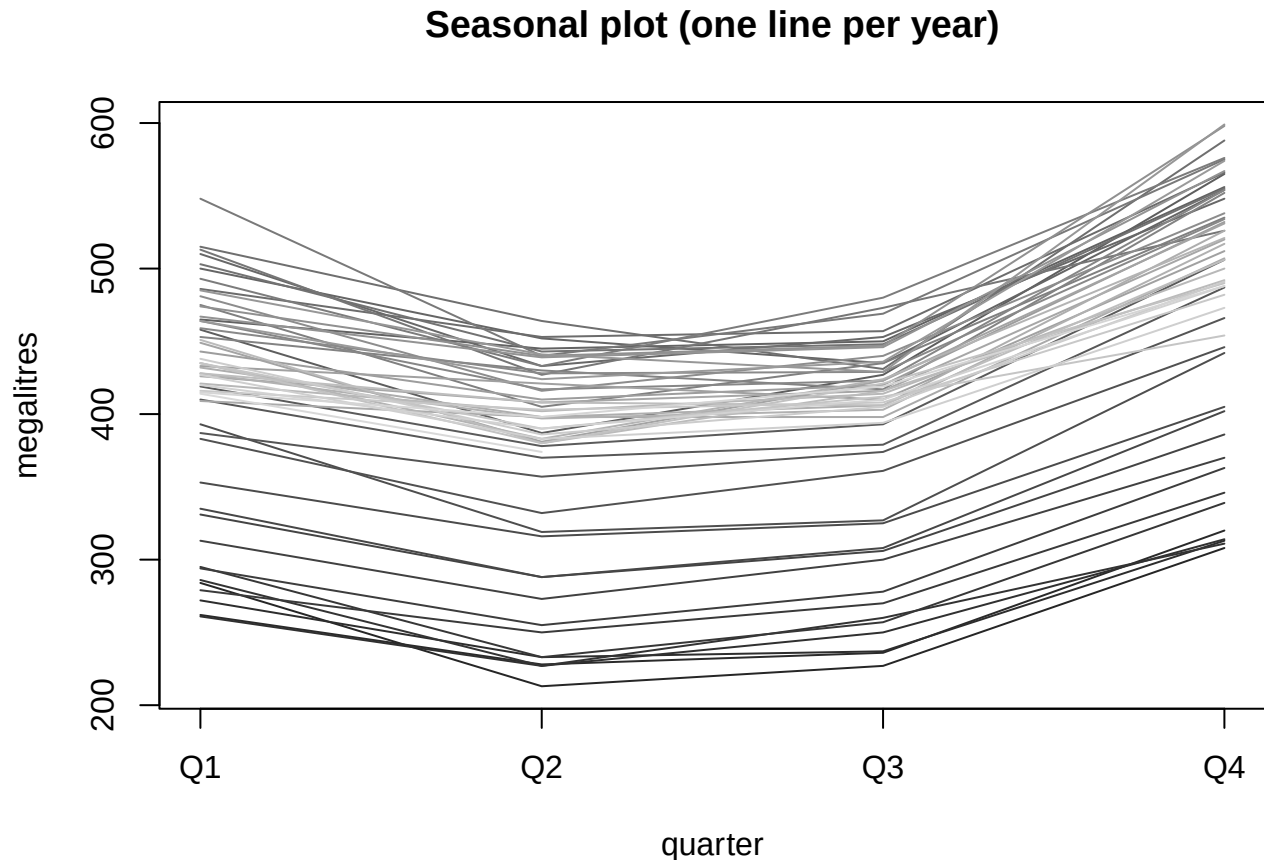


Figure 6: Seasonal plot: one line per year, quarters on the horizontal axis. Every year traces the same shape.

The ordinary time plot shows the seasonal wobble, but the years blur together and it is hard to compare the same quarter across years. The seasonal plot fixes this by drawing each year as its own line across the four quarters. What it reveals is how consistent the seasonal shape is: essentially every line follows the same path, down into Q2 and up to a peak in Q4. The lines are stacked by height, which is just the trend moving the whole year up or down, but the shape barely changes. It also makes an odd year easy to spot, because it

would be a line that does not follow the usual dip-then-peak track.

Q7: Seasonal subseries plot

```
monthplot(beer, ylab = "megalitres", xlab = "quarter",  
          main = "Seasonal subseries", col = "#2166ac", col.base = "red")
```

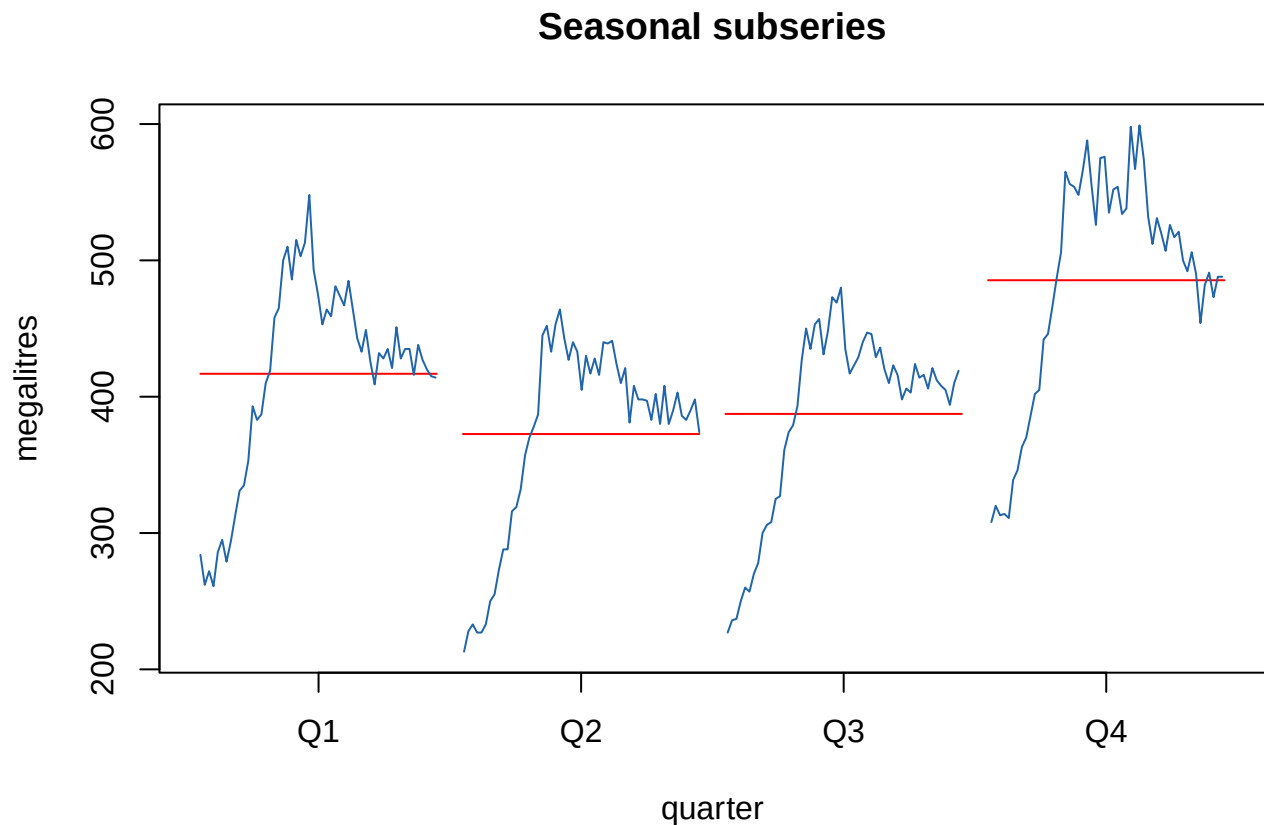


Figure 7: Seasonal subseries plot. Each panel is one quarter across all years; the red line is that quarter's mean.

This plot splits the series into four little series, one per quarter, and draws each quarter's mean as a red line. Reading off the red lines, **Q4 has the highest typical value and Q2 the lowest**, with Q1 second and Q3 third (means of about 485, 417, 387 and 373 for Q4, Q1, Q3, Q2). The seasonal pattern looks stable over time: within every panel the quarter follows the same overall shape as the whole series (up to the mid-1970s, a plateau, then a decline), and the ordering of the quarters never changes. The gap between the quarters is a little wider in the high-production middle decades and a little narrower recently, which matches the amplitude plot in Q4, but the pattern itself does not drift or flip.

Q8: Would you detrend or transform before modelling?

Two separate questions here, and my visual answers are different for each. I am not fitting anything, just arguing from the plots.

Detrend / account for trend and season: yes. The time plot in Q2 shows the average level moving a

lot over time (up, flat, then down), and the subseries plot in Q7 shows every quarter drifting in the same way. A series whose mean level changes like this is not stationary, so before modelling I would remove that structure, for example by taking a seasonal difference to knock out the strong quarterly pattern and a first difference to deal with the remaining slow movement in the level. The visual argument is simply that both the trend and the season are large and obvious, so a sensible model has to deal with them first.

Transform (log / Box-Cox): probably not. The amplitude plot in Q4 is the argument. If the variability grew clearly with the level, a log would be worth it, but here the swing does not scale cleanly with the level (the recent years are fairly high but have small swings). The variation is close enough to constant that a transformation would add complication for little gain. If I did anything at all it would be a very mild Box-Cox, but I would be comfortable leaving the series on its original scale.

In short: deal with the trend and the seasonality before modelling, but a variance transform is not really needed for this series.